

Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

Week-2

Week-2

The Librarian's Dashboard Scenario The library is now digital! The head librarian needs a report to see which members are active and which books are in the collection. You also need to handle a "Book Donation" process where a new book is added and assigned to a category at the exact same time—if one part fails, the whole process must stop to keep the catalog accurate.

Task to be Done

1. **The Catalog Search (JOINS):** Create a third table called Loans that connects member_id to book_id. Write a query to show the Member Name and the Book Title they have currently borrowed.
2. **Collection Stats (Aggregations):** Write a query to find the total number of books published in each year. o Hint: Use COUNT(book_id) and GROUP BY published_year.
3. **The Donation Transaction (ACID):** Use a Transaction (BEGIN...COMMIT) to add a new book to the Books table and simultaneously log the entry in a Donation_History table.
4. **Search Speed (Indexing):** Create an Index on the isbn column. Explain why searching by ISBN is now faster for the librarian than searching by Title. Skill Set Gained • Relational Logic: Connecting independent tables (Books + Members) through a Junction table (Loans). • Analytical SQL: Using GROUP BY to generate business insights. • Atomic Operations: Using Transactions to ensure multiple database changes succeed or fail as a single unit.

Solution:

Step 1 — Create Database

```
CREATE DATABASE bookflow_db;  
USE bookflow_db;
```



Fig 1: Workbench Action Output — green tick confirms success

Explanation:

- CREATE DATABASE builds a new empty database named bookflow_db — the container that will hold all four tables.
- USE bookflow_db activates it, so every following query runs inside this database.
- In Workbench, a green tick (✓) in the Action Output panel is the proof of successful execution — always point this out during viva/exam.

Step 2 — Create Books Table (DDL + Constraints)

```
CREATE TABLE Books (  
  book_id INT PRIMARY KEY,  
  title VARCHAR(100) NOT NULL,  
  isbn VARCHAR(20) UNIQUE,  
  published_year INT CHECK (published_year < 2027)  
);
```



Fig 2: Arrows mark the three data-integrity constraints

Explanation (each constraint):

- PRIMARY KEY (`book_id`) — uniquely identifies every book; automatically NOT NULL + UNIQUE.
- NOT NULL (`title`) — a book cannot be saved without a name.
- UNIQUE (`isbn`) — no two books can share the same ISBN, solving the "lost/duplicate book" problem.
- CHECK (`published_year < 2027`) — rejects any future year, preventing garbage data.

Step 3 & 4 — Insert Books and Display

```
INSERT INTO Books (book_id, title, isbn, published_year) VALUES  
(1, 'The Great Gatsby', '9780743273565', 1925),  
(2, 'To Kill a Mockingbird', '9780061120084', 1960),  
(3, '1984', '9780451524935', 1949);
```

```
SELECT * FROM Books;
```

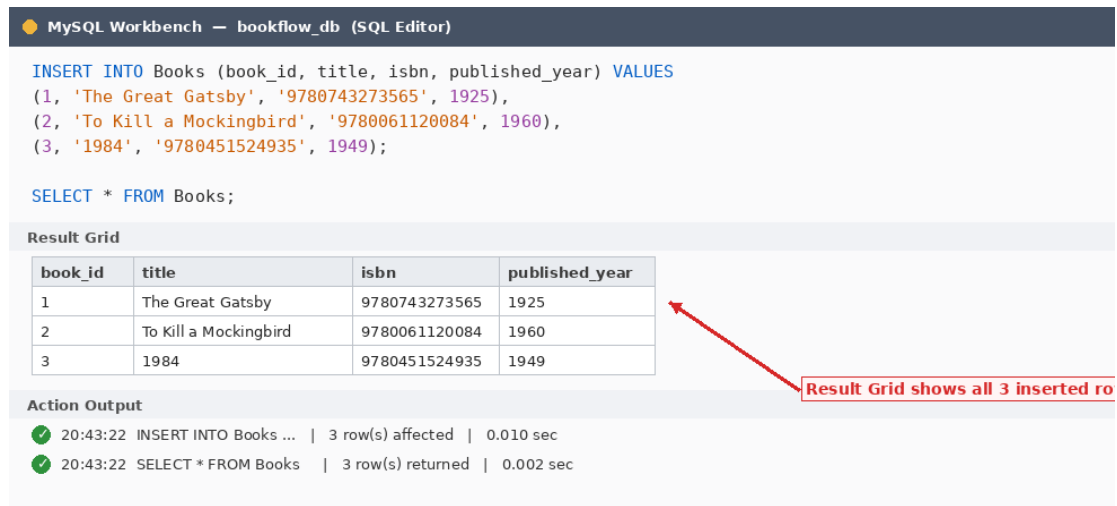


Fig 3: Result Grid displays the 3 inserted books

Explanation:

- One multi-row INSERT adds three books; the Action Output shows "3 row(s) affected".
- All rows pass every constraint — unique ISBNs, non-null titles, years below 2027.
- SELECT * confirms the data landed correctly in the Result Grid.

Step 5, 6 & 7 — Members Table: Create, Insert, Display

```
CREATE TABLE Members (  
  member_id INT PRIMARY KEY,  
  full_name VARCHAR(100),  
  email VARCHAR(100) UNIQUE  
);
```

```
INSERT INTO Members (member_id, full_name, email) VALUES  
(101, 'John Smith', 'john.smith@email.com'),  
(102, 'Emma Wilson', 'emma.wilson@email.com'),  
(103, 'Michael Brown', 'michael.brown@email.com');
```

```
SELECT * FROM Members;
```



The screenshot shows the MySQL Workbench interface. The SQL Editor at the top contains the SQL code for creating the Members table, inserting three rows, and selecting all data. Below the editor is the Result Grid, which displays the data from the SELECT statement. The Result Grid has three columns: member_id, full_name, and email. The data rows are: (101, John Smith, john.smith@email.com), (102, Emma Wilson, emma.wilson@email.com), and (103, Michael Brown, michael.brown@email.com). A red arrow points from a text box labeled "email column is UNIQUE — one account pe" to the email column header. The Action Output at the bottom shows the execution of the INSERT statement, indicating that 3 rows were affected in 0.008 seconds.

member_id	full_name	email
101	John Smith	john.smith@email.com
102	Emma Wilson	emma.wilson@email.com
103	Michael Brown	michael.brown@email.com

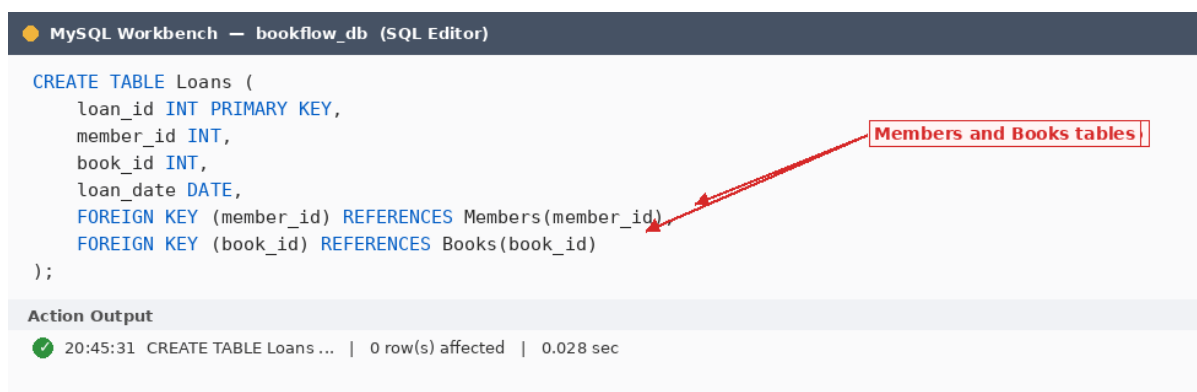
Fig 4: Members Result Grid — arrow highlights the UNIQUE email column

Explanation:

- PRIMARY KEY on member_id gives each member a unique identity (101, 102, 103).
- UNIQUE on email enforces one account per email — a second signup with john.smith@email.com would fail with ERROR 1062.
- This satisfies the rule: "no member can sign up without a valid email".

Step 8 — Create Loans Table (FOREIGN KEYS)

```
CREATE TABLE Loans (  
  loan_id INT PRIMARY KEY,  
  member_id INT,  
  book_id INT,  
  loan_date DATE,  
  FOREIGN KEY (member_id) REFERENCES Members(member_id),  
  FOREIGN KEY (book_id) REFERENCES Books(book_id)  
);
```



The screenshot shows the MySQL Workbench interface. The SQL Editor at the top contains the SQL code for creating the Loans table with foreign key constraints. Below the editor is the Action Output, which shows the execution of the CREATE TABLE statement, indicating that 0 rows were affected in 0.028 seconds. A red arrow points from a text box labeled "Members and Books tables" to the foreign key constraints in the SQL code.

```
CREATE TABLE Loans (  
  loan_id INT PRIMARY KEY,  
  member_id INT,  
  book_id INT,  
  loan_date DATE,  
  FOREIGN KEY (member_id) REFERENCES Members(member_id),  
  FOREIGN KEY (book_id) REFERENCES Books(book_id)  
);
```

Fig 5: Foreign keys connect Loans to the parent tables

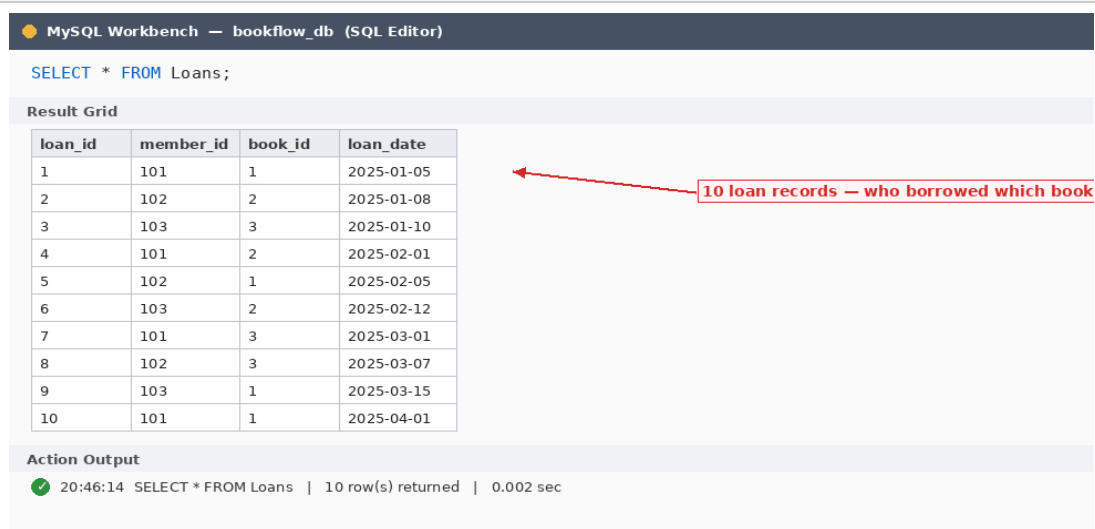
Explanation:

- Loans is the linking (junction) table — it records WHO borrowed WHICH book and WHEN. This directly solves the library's core problem.
- FOREIGN KEY (member_id) means a loan can only reference a member that actually exists in Members.
- FOREIGN KEY (book_id) means a loan can only reference a real book. This is referential integrity — no "ghost" loans.

Step 9 & 10 — Insert 10 Loans and Display

```
INSERT INTO Loans (loan_id, member_id, book_id, loan_date) VALUES
(1, 101, 1, '2025-01-05'), (2, 102, 2, '2025-01-08'),
(3, 103, 3, '2025-01-10'), (4, 101, 2, '2025-02-01'),
(5, 102, 1, '2025-02-05'), (6, 103, 2, '2025-02-12'),
(7, 101, 3, '2025-03-01'), (8, 102, 3, '2025-03-07'),
(9, 103, 1, '2025-03-15'), (10, 101, 1, '2025-04-01');
```

```
SELECT * FROM Loans;
```



MySQL Workbench — bookflow_db (SQL Editor)

```
SELECT * FROM Loans;
```

Result Grid

loan_id	member_id	book_id	loan_date
1	101	1	2025-01-05
2	102	2	2025-01-08
3	103	3	2025-01-10
4	101	2	2025-02-01
5	102	1	2025-02-05
6	103	2	2025-02-12
7	101	3	2025-03-01
8	102	3	2025-03-07
9	103	1	2025-03-15
10	101	1	2025-04-01

Action Output

20:46:14 SELECT * FROM Loans | 10 row(s) returned | 0.002 sec

Fig 6: All 10 loan transactions in the Result Grid

Explanation:

- Every member_id (101–103) and book_id (1–3) already exists in the parent tables, so all 10 inserts pass the FOREIGN KEY checks.
- If we tried member_id 999, MySQL would reject it: ERROR 1452 — a foreign key constraint fails.

Step 11 — JOIN Query (Who Borrowed What)

```
SELECT m.full_name AS Member_Name, b.title AS Book_Title
FROM Loans l
INNER JOIN Members m ON l.member_id = m.member_id
INNER JOIN Books b ON l.book_id = b.book_id;
```

MySQL Workbench — bookflow_db (SQL Editor)

```
SELECT m.full_name AS Member_Name, b.title AS Book_Title
FROM Loans l
INNER JOIN Members m ON l.member_id = m.member_id
INNER JOIN Books b ON l.book_id = b.book_id;
```

ON clause matches the ID column

Result Grid

Member_Name	Book_Title
John Smith	The Great Gatsby
Emma Wilson	To Kill a Mockingbird
Michael Brown	1984
John Smith	To Kill a Mockingbird
Emma Wilson	The Great Gatsby
Michael Brown	To Kill a Mockingbird
John Smith	1984
Emma Wilson	1984
Michael Brown	The Great Gatsby
John Smith	The Great Gatsby

IDs replaced by readable names & titles

Action Output

20:47:02 SELECT ... INNER JOIN ... | 10 row(s) returned | 0.003 sec

Fig 7: INNER JOIN replaces numeric IDs with readable names and titles

Explanation:

- The Loans table only stores IDs (101, 1). INNER JOIN pulls the matching full_name from Members and title from Books using the ON conditions.
- Table aliases (l, m, b) shorten the query; AS renames the output columns to Member_Name and Book_Title.
- 10 loans in → 10 readable rows out. This is the librarian's answer to "who has which book?".

Step 12 — GROUP BY Query (Books per Year)

```
SELECT published_year, COUNT(book_id) AS Total_Books
FROM Books
GROUP BY published_year
ORDER BY published_year;
```

MySQL Workbench — bookflow_db (SQL Editor)

```
SELECT published_year, COUNT(book_id) AS Total_Books
FROM Books
GROUP BY published_year
ORDER BY published_year;
```

One row per year — COUNT() aggregates books

Result Grid

published_year	Total_Books
1925	1
1949	1
1960	1

Action Output

20:48:20 SELECT ... GROUP BY ... | 3 row(s) returned | 0.002 sec

Fig 8: One aggregated row per publication year

Explanation:

- GROUP BY collapses rows that share the same published_year into one group.
- COUNT(book_id) counts the books inside each group; ORDER BY sorts years ascending.
- Exam rule: every non-aggregated column in SELECT (published_year) must appear in GROUP BY.

Step 13–16 — Donation via TRANSACTION (Atomicity)

```
CREATE TABLE Donation_History (  
  donation_id INT PRIMARY KEY,  
  book_id INT,  
  donor_name VARCHAR(100),  
  donation_date DATE,  
  FOREIGN KEY (book_id) REFERENCES Books(book_id)  
);  
  
START TRANSACTION;  
  
INSERT INTO Books (book_id, title, isbn, published_year)  
VALUES (4, 'Animal Farm', '9780451526342', 1945);  
  
INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date)  
VALUES (1, 4, 'Raj Kumar', CURDATE());  
  
COMMIT;
```

MySQL Workbench — bookflow_db (SQL Editor)

```
START TRANSACTION;  
  
INSERT INTO Books (book_id, title, isbn, published_year)  
VALUES (4, 'Animal Farm', '9780451526342', 1945);  
  
INSERT INTO Donation_History (donation_id, book_id, donor_name, donation_date)  
VALUES (1, 4, 'Raj Kumar', CURDATE());  
  
COMMIT;
```

Action Output

✓	20:50:11	START TRANSACTION		0 row(s) affected		0.000 sec
✓	20:50:11	INSERT INTO Books ...		1 row(s) affected		0.005 sec
✓	20:50:11	INSERT INTO Donation_...		1 row(s) affected		0.004 sec
✓	20:50:11	COMMIT		0 row(s) affected		0.009 sec

Fig 9: All four statements succeed — COMMIT makes both inserts permanent together

Output tables after COMMIT:

donation_id	book_id	donor_name	donation_date
1	4	Raj Kumar	2025-07-14

book_id	title	isbn	published_year
1	The Great Gatsby	9780743273565	1925
2	To Kill a Mockingbird	9780061120084	1960
3	1984	9780451524935	1949
4	Animal Farm	9780451526342	1945

Explanation:

- A donated book must appear in BOTH Books and Donation_History — never in just one. START TRANSACTION groups the two inserts into one atomic unit.
- COMMIT saves both permanently. If anything failed mid-way, ROLLBACK would undo everything — this is the A (Atomicity) in ACID.
- CURDATE() automatically stamps today's date (2025-07-14) as the donation date.
- Note: Workbench has auto-commit ON by default; START TRANSACTION temporarily overrides it until COMMIT/ROLLBACK.

Step 17 & 18 — Index on ISBN + Fast Search

```
CREATE INDEX idx_books_isbn ON Books(isbn);
```

```
SELECT * FROM Books WHERE isbn = '9780451524935';
```

MySQL Workbench — bookflow_db (SQL Editor)

```
CREATE INDEX idx_books_isbn ON Books(isbn);
```

```
SELECT * FROM Books WHERE isbn = '9780451524935';
```

WHERE on isbn now uses the index (fast)

Result Grid

book_id	title	isbn	published_year
3	1984	9780451524935	1949

Exact match found instantly

Action Output

```
20:51:44 CREATE INDEX idx_books_isbn | 0 row(s) affected | 0.041 sec
20:51:44 SELECT * FROM Books WHERE ... | 1 row(s) returned | 0.001 sec
```

Fig 10: WHERE clause on isbn uses the index for an instant lookup

Explanation:

- An index is like a book's alphabetical index — instead of scanning every row (full table scan), MySQL jumps straight to the matching ISBN (B-tree lookup).
- On 4 rows the difference is invisible, but on 4 million books this turns seconds into milliseconds.
- The search returns exactly one row: book_id 3, "1984" — proving the lookup works.
- Exam tip: the UNIQUE constraint on isbn already created an internal index; idx_books_isbn demonstrates explicit index creation as asked.

Final State of All Tables

Books

book_id	title	isbn	published_year
1	The Great Gatsby	9780743273565	1925
2	To Kill a Mockingbird	9780061120084	1960
3	1984	9780451524935	1949
4	Animal Farm	9780451526342	1945

Members

member_id	full_name	email
101	John Smith	john.smith@email.com
102	Emma Wilson	emma.wilson@email.com
103	Michael Brown	michael.brown@email.com

Loans

loan_id	member_id	book_id	loan_date
1	101	1	2025-01-05
2	102	2	2025-01-08
3	103	3	2025-01-10
4	101	2	2025-02-01
5	102	1	2025-02-05
6	103	2	2025-02-12
7	101	3	2025-03-01

loan_id	member_id	book_id	loan_date
8	102	3	2025-03-07
9	103	1	2025-03-15
10	101	1	2025-04-01

Donation_History

donation_id	book_id	donor_name	donation_date
1	4	Raj Kumar	2025-07-14

Exam Cheat Sheet

Concept	Where Used	One-Line Answer for Viva
PRIMARY KEY	All 4 tables	Unique row identity; NOT NULL + UNIQUE combined
NOT NULL	Books.title	Column can never be empty
UNIQUE	isbn, email	No duplicates allowed in the column
CHECK	published_year < 2027	Value must satisfy the condition
FOREIGN KEY	Loans, Donation_History	Child value must exist in the parent table
INNER JOIN	Step 11	Returns only rows that match in both tables
GROUP BY + COUNT	Step 12	Aggregates rows into groups and counts them
TRANSACTION	Step 14	All-or-nothing: COMMIT saves, ROLLBACK undoes
INDEX	Step 17	B-tree lookup — fast WHERE search on big tables

How to Run in MySQL Workbench

- Open Workbench → connect to Local instance → File → New Query Tab.
- Paste each step's query → press the lightning-bolt (⚡) button or Ctrl+Enter to execute.
- Read results in the Result Grid; confirm success via the green tick in Action Output.
- For the exam: show the green ticks, the Result Grids, and explain each arrow-marked constraint.

Viva Questions and Answers

1. What is the purpose of the Loans table in the BookFlow database?

Answer:

The **Loans** table acts as a junction (bridge) table between the **Members** and **Books** tables. It stores borrowing transactions and helps track which member has borrowed which book and on what date.

2. Why do we use JOIN operations in SQL?

Answer:

JOIN is used to combine data from two or more related tables based on a common column. In BookFlow, JOIN is used to display member names and book titles together by connecting the Members, Loans, and Books tables.

3. What is the purpose of the GROUP BY clause?

Answer:

The **GROUP BY** clause groups rows that have the same values in a specified column. It is commonly used with aggregate functions such as **COUNT()**, **SUM()**, and **AVG()**. In this project, it is used to count the number of books published in each year.

4. What is a Transaction, and why is it important?

Answer:

A **Transaction** is a sequence of SQL operations treated as a single unit of work. It ensures that all operations either complete successfully (**COMMIT**) or none are applied (**ROLLBACK**) if an error occurs. This maintains data consistency and integrity.

5. Why is an Index created on the ISBN column?

Answer:

An **Index** is created on the ISBN column to improve search performance. Since ISBN values are unique and frequently searched, the database can locate records much faster using the index instead of scanning every row in the Books table. This reduces query execution time and improves efficiency.

Assignments

Problem Statement 1: SkyTrack Airline Management System

A regional airline company currently manages its flight schedules, passenger records, and ticket bookings using spreadsheets. As the number of flights and passengers continues to grow, it has become difficult to track reservations, maintain accurate records, and generate operational reports. To improve efficiency, the airline plans to develop a database system called **SkyTrack**. As a database developer, your task is to create a database named **skytrack_db** and design three tables: **Flights**, **Passengers**, and **Bookings**. The Flights table should contain details such as **flight_id**, **flight_number**, **source**, **destination**, **departure_date**, and **ticket_price**, while the Passengers table should store **passenger_id**, **passenger_name**, and **email**. The Bookings table should connect passengers and flights using **passenger_id** and **flight_id** as foreign keys. Apply appropriate constraints, including **PRIMARY KEY**, **UNIQUE**, and **CHECK** constraints to ensure data integrity. Insert at least **10 records** into each table. Display all table contents and write an SQL query using **INNER JOIN** to show the passenger name, flight number, source, and destination for every booking. Generate a report showing the total number of flights operating to each destination using **COUNT()** and **GROUP BY**. Create a **Flight_History** table and demonstrate the use of a **Transaction** by adding a new flight and simultaneously recording the entry in the history table, ensuring both operations succeed or fail together. Finally, create an **Index** on the **flight_number** column and explain why searching by flight number is faster than searching by destination. This exercise develops skills in relational database design, JOIN operations, aggregate functions, transactions, and indexing.

Problem Statement 2: MediCare Hospital Management System

A hospital currently maintains patient records, doctor information, and appointment schedules manually, which often results in scheduling conflicts, misplaced records, and delays in accessing important information. To modernize its operations, the hospital wants to develop a database system called **MediCare HMS**. As a database developer, your task is to create a database named **medicare_db** and design three tables: **Doctors**, **Patients**, and **Appointments**. The Doctors table should store details such as **doctor_id**, **doctor_name**, **specialization**, and **consultation_fee**, while the Patients table should contain **patient_id**, **patient_name**, and **email**. The Appointments table should connect doctors and patients using foreign keys and include an appointment date. Apply appropriate constraints such as **PRIMARY KEY**, **UNIQUE**, and **CHECK** constraints to ensure data accuracy and consistency. Insert at least **10 records** into each table. Display all table contents and write an SQL query using **INNER JOIN** to show the patient name, doctor name, specialization, and appointment date for all appointments. Generate a report displaying the number of doctors available in each specialization using **COUNT()** and **GROUP BY**. Create a **Doctor_History** table and demonstrate the use of a **Transaction** by registering a new doctor and simultaneously recording the registration in the history table, ensuring both operations are completed successfully as a single unit. Finally, create an **Index** on the **specialization** column and explain how indexing improves search

performance when locating doctors by specialization. This exercise helps students understand database design, table relationships, SQL queries, aggregate functions, transactions, and query optimization techniques.